

Practical Machine Learning

- What is Machine Learning?
- ML is a branch of AI that allows computers to learn from data and make predictions or decisions without being explicitly programmed.
- Its like teaching a system by giving it examples instead of writing rules.

- Types of Machine Learning

1. Supervised Learning

- type of ML where the model is trained on a labeled dataset, meaning:

- Input (x) : Features (eg - age, income)
- Output (y) : known correct label (eg - will buy or not)

- The model learns the relationship between input and output, so that it can predict the output for new/unseen inputs.

- The goal is to learn a function $(x) \rightarrow y$ from the training data so that it can predict y for any new x.

- Unsupervised Learning

- type of ML where the model is given only input data (x) without any labeled output (y).

- The goal is to find hidden patterns, groupings, or structures in the data - without knowing what the 'right answer' is.

- Why Unsupervised?

- Because ~~no~~ no one tells the machine what the correct answers are - the algorithm tries to understand the structure on its own.

- Reinforcement Learning

- type of ML where an agent learns to make decisions by interacting with an environment to achieve a goal.

- The agent takes actions.

- The environment gives rewards or penalties.

- Agent learns to maximize total reward over time.

- Its like trial and error learning - just like how humans learn to ride a cycle, play a game or drive.

Train - Test - Split

- `train_test_split()` is a function from `Scikit-Learn` used to split your dataset into two or more parts.
- Training set : Used to train the ML model.
- Testing set : Used to test model performance.
- We select a particular amount of rows from the dataset along with the predictor and response columns.
- We split the dataset into 4 parts.
 - x_train : these are rows and columns of the predictors.
 - y_train : these are rows of the response column.
- # `x_train` and `y_train` together are the training set and used to train the model.
- # After the model is trained, we use `x-test` to check the accuracy of the model.
- # Using `x-test`, the model will predict output `y-pred` which is compared with the actual output `y-test`.

While testing, only x-test is given to the model, y-test is not provided.

x-test and y-test together are the testing set.

You must split the data, otherwise your model will 'cheat' by memorizing the answers instead of learning general patterns.

Why Do We Split?

- Imagine you're a student:

- You study from a book (training set)

- You give an exam based on questions you've never seen (test set)

- That's how we judge if the student has really understood the topic.

- Same logic with ML - if the model performs well on unseen ~~data~~ test data → it's genuinely good.

$x_{\text{train}}, x_{\text{test}}, y_{\text{train}}, y_{\text{test}} = \text{train_test_split}(x, y, \text{test_size}=0.2)$

- $x \rightarrow$ predictor columns.
- $y \rightarrow$ response column.
- $\text{test_size} \rightarrow 0.2 \rightarrow$ meaning the total dataset is split into 80% and 20% parts. 20% (0.2) goes to test set.
- $\text{random_state} = 42 \rightarrow$ keeps result same every time you run (reproducibility).
- $\text{shuffle} = \text{True} \rightarrow$ shuffles the data before splitting (default = True).
- $\text{stratify} = y \rightarrow$ keeps same class ratio in both sets (useful in classification).

steps

1. Import clean dataset.
2. Split into $x_{\text{train}}, x_{\text{test}}, y_{\text{train}}, y_{\text{test}}$.
3. Using $x_{\text{train}}, y_{\text{train}}$, create the model.
4. Pass only x_{test} to the model to generate y_{pred} .
5. Compare y_{pred} with y_{test} .

PAC - Probably Approximately Correct

- it is a theoretical framework in ML (proposed by Leslie Valiant in 1984) that tries to answer the question:

"How much data do we need so that a machine learning algorithm can learn well enough?"

- Approximately: how close you get to the correct result.

- Probably: how often you get that close to the correct result.

So in short: "A model is PAC if, with high probability, it gives an answer that's close enough to the truth."

Occam's Razer

"The simplest explanation that fits the data is usually the best one."

In ML, when you have multiple models that all perform well on training data, prefer the one that's simplest.

No Free Lunch Theorem

- According to NFL, no one ML Algorithm works best for every problem. Meaning, there is no universal best model. You can't just blindly apply one algorithm to every dataset and expect great results.
- Therefore, we have to attempt many models to discover the one that performs best for a specific ~~column~~ problem.

Validation of Data

- Data validation is a critical step in the machine learning pipeline. It ensures that the input data is correct, consistent and suitable for training models. Think of it as quality control for your data before training.
- ~~How is it performed?~~
- ~~Training data is split internally into another 80% - 20%.~~
- ~~20% will be test data for during validation.~~

- How is it performed?

- It is performed on Train data.

- This data is divided into no. of parts i.e.

subset and each subset is hidden each time and remaining part is given to train the model.

- This subset is called Validation Data.

- K-Fold

- In training data, split the dataset into k equal parts.

- Each fold is used as validation set and use the remaining data as training set.

- Each fold will become the validation set only once.

- Train the model on training set and evaluate on validation set.

- Bias
- Bias means the error due to overly simplistic assumptions in the learning algorithms.
- Bias is the error associated with the training set.

- Algorithmic Bias

- Its how far off your model's predictions are from the actual values, on average.
- Algorithmic Bias
- When the model is too simple to capture the true patterns in data.
- Eg : Using Linear Regression to fit a non-linear relationship.

- Data Bias (Training Bias)

- When the training data doesn't represent the actual real-world scenario.

- Eg : A model trained only on data

- Prejudice or Labeling Bias

- Happens when humans introduce bias while labelling the data.
- Eg : A dataset with biased labels based on stereotypes.

If Bias is high; your model will:

- make strong assumptions about the data.
- misses important relationships.
- performs poorly even on the training data.
- is said to be underfitting.

- Variance

- Variance refers to how sensitive a model is to small changes in the training data.
- A model with high variance will learn the training data very well (maybe too well), but it won't perform well on new/unseen data.

- High Variance Causes

- model is too complex.
- Not enough training data.
- Noise in the data.
- No regularization.
- How to reduce Variance?
- Use a simpler model.
- Use more training data.
- Apply regularization.
- Use cross-validation to test generalization.

- Overfitting

- Overfitting happens when a machine learning model performs very well on training data but poorly on unseen / test data.

- Real - Life Analogy

- You memorize all the practice questions word-for-word.
- But in the actual exam, questions are different.
- You panic because you didn't understand the concepts, you just memorized.

- Why does Overfitting happen?

- Model is too complex.
- Too few training examples.
- Noisy or irrelevant data.
- Not using regularization.
- Too many epochs in training (in deep learning).

- How to detect Overfitting?

- High accuracy on training data.
- Low accuracy on test/validation data.
- Big gap between training and validation loss.

- Underfitting

- happens when a model is too simple to learn the underlying patterns in the data.
- model performs poorly on training data and obviously on test data as well.
- Real Life Analogy

- You read a 1-page summary and go for an exam.

- You don't learn the concepts properly.

- So you fail both in practice tests (training) and the final exam (test data).

- Why does Underfitting happen?

- model is too simple.

- too few features used.

- not enough training

- incorrect assumptions

- too much regularization.

- Signs of Underfitting

- low training accuracy - model can't even learn from known data.

- low test accuracy - model performs bad on unseen data too.

- high bias, low variance - model is too restricted.

- Simple Linear Regression

- It is a statistical method that finds the best straight line ($y = mx + c$) to predict the value of one variable based on another.

- used when we have :

- one predictor (continuous)

- one response (continuous)

$$y = mx + c$$

$y \rightarrow$ predicted (dependent) value. $\rightarrow$ response

$x \rightarrow$ input (independent) value $\rightarrow$ predictor.

$m \rightarrow$ slope of line (how much y changes with x)

$c \rightarrow$ y-intercept (y when $x = 0$)

Say you have dataset with one predictor and one response column, both continuous.

Eg - CGPA $\rightarrow$ predictor

Package $\rightarrow$ response

Linear Regression helps us to find out Package for a particular CGPA.

In reality, our data is never linear, hence the

output y of the equation is called predicted value which is close to the actual value.

- Multiple Linear Regression -

- Both simple and multiple linear regressions assume that there is a linear relationship between the input variable and output target variable.

- The only difference between the two is that multiple linear regression takes multiple predictors.

$$y = m_1 x_1 + m_2 x_2 + \dots + m_n x_n + c$$

- Training a Linear Regression Model

- We train a LR model with a method called Ordinary Least Squares (OLS). The goal of training is to find the weights (coefficients) in the linear equation $y = mx + c$.

- Random Weight Initialization:

- In practice, m, c are unknown at the beginning. The goal of the procedure is to find the appropriate values for these parameters.

To start the process, we set the values of the weights at random or just initialize with 0.

- Input the initialized weights ~~me~~ into the linear equation and generate a prediction for each observation point.
 - Calculate the Residual Sum of Squares (RSS). Residuals, or error terms, are the difference between each actual output and predicted output.
 - We square the residuals for each observation point and sum the residuals to reach our RSS.
- # The basis here is that a lower RSS means that our line of best fit comes closer to each data point and vice versa.
- Machine Learning approaches find the best parameters for the linear model by defining a 'cost function' and minimizing it via gradient descent. By doing so, we obtain the best possible values for the weights.
 - Cost Function
 - In case of LR, the cost function is the same as the residual sum of errors. The algorithm solves the minimization problem and is achieved using Gradient Descent.

- Gradient Descent

- it is the method of changing weights based on the loss function for each data point.

We calculate the sum of squared errors at each ~~point~~ input-output data point.

- Loss Function

- 'Loss' in ML helps us understand the diff between the predicted value & actual value.

- The function used to quantify this loss during the training phase in the form of a single real number is known as the 'Loss Function'.

Loss Function : Used when we refer to the error for a single training example.

Cost Function : Used to refer to an average of the loss functions over an entire training data.

- Mean Squared Error (MSE)

- MSE is computed as RSS divided by the total number of data points, i.e., the total number of observations or examples in our given dataset. MSE ~~test~~ tells us what the average RSS is per data point.

$$MSE = \frac{1}{n} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right]$$

n = no. of observations

y_i = actual value

$\hat{y}_i$ = predicted value

- Root Mean Squared Error (RMSE)

- RMSE takes the MSE value and applies a square root over it. RMSE can be directly used to interpret the 'average error' that our prediction model makes.

$$RMSE = \sqrt{MSE}$$

- Offset

- A fixed value that is added (or sometimes subtracted) from something - like from a prediction, feature or a formula.
- It is not learned by the model - it's something you already know, and you give it directly to the model.

- Eg: Imagine you are predicting salary of employees.

- You already know that every employee gets 10000 as fixed bonus - ~~no~~ no matter what. This 10000 is an offset, which is always added.

- So the model learns the rest (based on experience, skills etc) but you already add 10000 to the prediction.

- Ordinary Least Squares (OLS)

- OLS is a method used in Linear Regression to find the best line that fits the data.
- OLS finds the straight line that minimizes the total error between the actual data points and the predicted line.

Pros

- Easy to understand and implement
- Works well for linear data.
- Fast and efficient for small datasets.

Limitations

- Doesn't work well if data is non-linear.
- Sensitive to outliers.
- Assumes that errors are normally distributed and independent.

- Categorical Regression

- Regression where some of the features (predictors) or sometimes even the target (response) are categorical.
- Python does not understand labels such as Male, Female, Mumbai, Delhi etc, so we need to encode them (convert them to numbers first) before regression.
- This method is called One-Hot Encoding where we convert categorical values (text) into numeric values so that machine learning model can understand them.

- For example, we have a category named 'City' with Mumbai, Delhi, Pune in it.
- Python will assign 0 to the city that comes alphabetically first, i.e., Delhi, and 1 to the rest.
- All cities will get a separate column.

City → ['Mumbai', 'Delhi', 'Pune']

City-Mumbai

City-Delhi

City-Pune

1
0
0

0
1
0

0
0
1

- This means, wherever there is 1, that row belongs to that particular city.

- 'Delhi' being alphabetically first, is the Reference Value and all other cities are with respect to 'Delhi'.

- Logistic Regression

- In Logistic Regression, the model predicts the probability that specific outcome occurs. For example, given someone's financial profile, we might predict the probability that their loan is approved. The output of the model is a value between 0 and 1. Based on a threshold - often 0.5, we classify the outcome as either "approved" or "not-approved".

- Because we cannot model probabilities directly by using a linear function - because probabilities are constrained between 0 and 1 - we instead work with odds.

- Probability: measures the chance of an event occurring out of all possible outcomes.

- Odds: Odds compare the chance of an event occurring to the chance of it not occurring.

- If p is the probability of an outcome, then

$$\text{odds} = \frac{p}{1-p}$$

- However the odds are not symmetric around

1. For example,

- To address this imbalance, we take the

log

algorithm of the odds, which transforms the unbounded $[0, \infty)$ scale of odds to the real

number line $(-\infty, \infty)$. This is known as

log of odds or logit and is the

foundation of the logistic regression model.

$$\log \left[\frac{p}{1-p} \right]$$

- Logistic Regression does not model y directly - it models the probability of y through the log of

odds. Because of this, we cannot say that a

one-unit increase in x results in a constant

unit change in y . Instead, we interpret the

coefficient in terms of its effects on the log

of odds, and by extension, on the probability of the outcome.

- In Logistic Regression:

- a positive coefficient means the log of odds

of the outcome increases as the input increases

and hence probability of the outcome occurring increases.

- A negative coefficient means log of odds decreases as input increases, hence, the probability of the outcome occurring decreases.
- A zero coefficient means the input has no effect on the outcome.

- Binary Logistic Regression

- a ML algorithm used to classify data into two categories like Yes/No, 0/1, True/False.
- Response will either be True or False, i.e., it will be the probability of an event happening.
- Nominal Logistic Regression
- used when the target variable (response) is categorical with more than two classes and the classes have no natural order.
- It is also called Multinomial Logistic Regression.
- Example :

* Input (x)

- Age, Salary, Location

- Exam Score, Study Time

Output (Y)

Product Type : A/B/C

Grade : A/B/C.

- How Nominal Regression Works?

- For each class k , the model estimates the log of odds of class k against a reference class.

- Example: Predicting among 3 classes (A, B, C):

- Choose class A as base (reference).

- Model predicts:

• $\log(P(B)/P(A))$

• $\log(P(C)/P(A))$

- Pros:

- Works well for multiclass problems.

- Gives probabilities per class.

- Interpretable co-efficients.

- Cons

- Assumes independent classes (no overlap)

- Needs one class as reference/base.

- Doesn't model class order (if present).

- Ordinal Logistic Regression

- used when the response (Y) has multiple categories with a natural order, but the distance between them is not fixed.

- Example:

- Customer Satisfaction : Bad < Average < Good.

- Ratings : 1 star < 2 < 3

- How it works?

- It uses a technique called Cumulative Logit Model.

- Step 1:

Model predicts probabilities of being in a class less than or equal to each ~~other~~ category.

- Step 2:

Applies log of odds and estimates thresholds between ordered classes.

- Pros:

- Perfect for ordered categories.
- Better than treating ordered classes as nominal.
- Gives interpretable thresholds.

- Cons:

- Can be complex to interpret.
- Assumes proportional odds (same relationship across all thresholds).

- Sigmoid Function

- The sigmoid function is a mathematical function that takes any real-valued number (+ve, -ve or zero) and maps it to a value between 0 and 1.
- It's commonly used in binary classification models, especially logistic regression and neural networks to represent probabilities.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$x = \text{input } (-\infty, \infty)$

$e = \text{Euler's number} \approx 2.718$

$\sigma(x) = \text{output, ranging between 0 and 1.}$

- In Logistic Regression, the model models the probability of a binary class label (0 or 1). The output from the linear part ($y = mx + c$) is passed through the sigmoid:

$$p = \sigma(x) = \frac{1}{1 + e^{-x}}$$

If $p \geq 0.5$, classify as Class 1.

$p < 0.5$, classify as Class 0.

- Sigmoid is also used as an activation function in Neural Networks, especially in the output layer when solving binary classification problem.

It helps squashing outputs between 0 and 1, so the model can learn probabilities.

- Feature Scaling

- technique to standardize the range of independent variables so that they contribute equally to the model's performance.

- especially useful when the models are created based on the distance between data points.

- Why feature scaling?

- Different features can have different units and scales

Age : 0 - 100

Salary : 0 - 100000

Height : 0 - 250

- If not scaled :

- features with larger values dominate others.

- It slows down gradient descent convergence.

- Distance based algorithms give biased results.

- Advantages:

- coefficients are easier to interpret.

- convergence is faster

- model will not be inclined towards the bigger magnitude variables.

- best used when you want values to stay in a bounded range. Only between 0 and 1.

- Min-Max Scaler

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Use : Neural Networks, image data.

- Standardization (Z-score Normalization)

μ = mean

σ = std dev

$$x' = \frac{x - \mu}{\sigma}$$

Use : Linear models, PCA, logistic regression, SVM.

- Robust Scaling

used when we have outliers.

$$x' = (x - \text{median}) / \text{IQR}$$

- MaxAbs Scaler

$$x' = \frac{x}{|x_{\max}|}$$

- StandardScaler()

- It is a preprocessing tool from scikit-learn used to standardize features by removing the mean and scaling to unit variance. It transforms the data such that each feature has:

mean = 0

std. dev = 1

- For each feature x :

$$z = \frac{x - \mu}{\sigma}$$

μ = mean of the feature

σ = standard deviation.

- When to use :

- Use when your model is sensitive to feature scaling.

- When you want to normalize continuous variables.

- Variance Inflation Factor (VIF)

- VIF tells us how much one feature (column) is related to the other features in your dataset.
- VIF shows how much a feature is 'inflated' due to multicollinearity.

- Why use VIF?

- In regression, we assume that features are independent or not too correlated.
- If two or more features are strongly related, it can confuse the model and make the results unreliable.
- VIF helps us detect such multicollinearity.

- Grid Search CV

- is a tool in scikit-learn used to find the best combination of parameters for your machine learning model.
- Think of it like: "Trying all possible combinations of settings (parameters) to see which one gives the best accuracy or result."

- Lasso Regression (L1) regularization

- Lasso Regression, also known as L1 regularization, is a type of linear regression that adds a penalty term to the cost function to shrink or eliminate some of the coefficients of the model.

- The penalty term is the absolute value of the coefficients multiplied by a scalar value, called regularization parameter.

- LASSO stands for 'Least Absolute Shrinkage and Selection Operator'.

- In Lasso, the cost function is defined as the mean square error of the model plus the penalty term, which is the sum of the absolute values of the coefficients multiplied by the regularization parameter. The goal is to minimize this cost function by adjusting the coefficients of the model.

$$\text{cost function} = \frac{1}{2} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right] + \alpha \sum_{j=1}^p |\beta_j|$$

y_i = actual value

$\hat{y}_i$ = predicted value

β_j = coefficients

α = regularization parameter.

- Lasso Regression has the added benefit of feature selection, as it tends to shrink coefficients of less important features to zero, effectively eliminating them from the model.

- Ridge Regression ()

- Ridge is very similar to Lasso except for the following :

- L2 penalty.

$$\text{cost function} = \frac{1}{2n} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right] + \frac{\lambda}{2} \sum_{j=1}^p (\beta_j)^2$$

- Ridge shrinks coefficients closer to 0 but not exactly to 0.

- Does not help in feature selection.

- used when you want to reduce overfitting but also keep all features.

- Elastic Net Regression

- it combines the features of both Lasso and Ridge regression.
- it is a regularized regression technique that is used to deal with the problems of multicollinearity and overfitting.
- Like Ridge, it shrinks ~~the~~ coefficients to reduce overfitting.
- Like Lasso, it can set some coefficients to 0 for feature ~~select~~ selection.

$$\text{cost function} = \frac{1}{2n} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right] + \alpha \left[\lambda \sum_{j=1}^p |\beta_j| + (1 - \lambda) \sum_{j=1}^p (\beta_j)^2 \right]$$

α = regularization parameter.

λ = ~~the~~ λ_1 -ratio = controls the mix between Lasso and Ridge

$\lambda = 1 \rightarrow$ pure Lasso

$\lambda = 0 \rightarrow$ pure Ridge

Between 0 and 1 $\rightarrow$ a mix of both

- Advantages :

- Feature Selection : can perform feature selection by shrinking the coefficients of irrelevant variables to zero. This results in a model with fewer variables, which is easier to interpret.

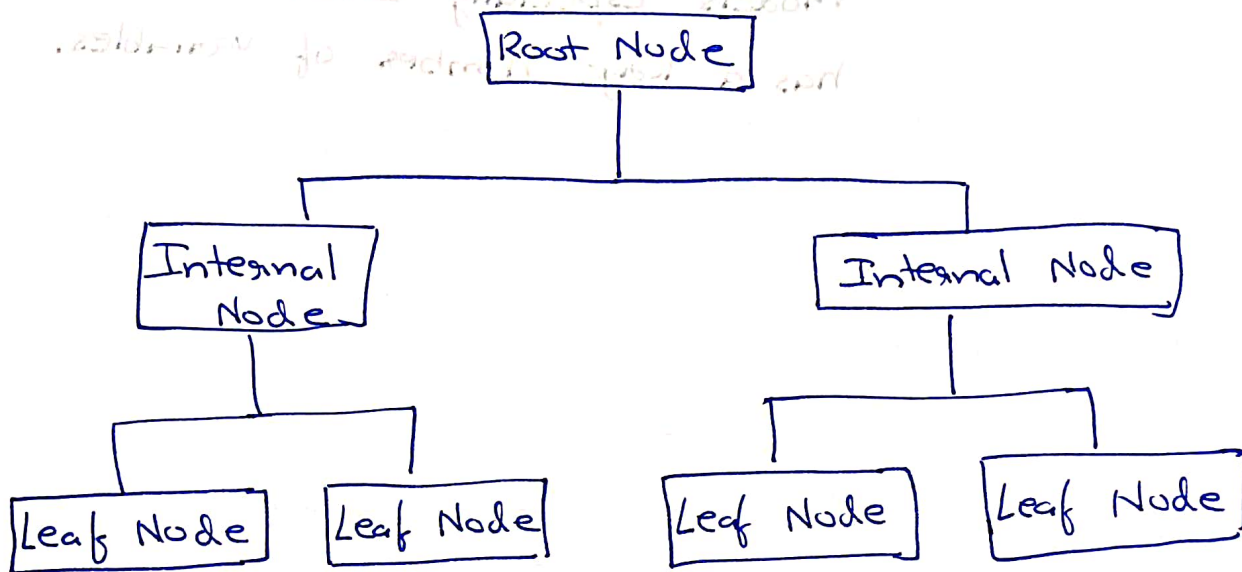
- Robustness: it is more robust than other linear regression ~~techn~~ techniques such as Ridge, Lasso as it combines the strength of both techniques. It can handle correlated variables and variables with different scales.

- Better performance: it performs better than other models especially when the dataset has a large number of variables.



- Decision Tree

- A decision tree is a supervised learning algorithm used for classification and regression tasks.
- It works like a flowchart where each internal node represents a feature, each branch represents a decision rule and each leaf node represents the outcome (label).



- Advantages

- Easy to understand and visualize.
- No need for feature scaling.
- Can handle both numeric and categorical data.
- Captures non-linear relationships.
- ~~Per~~ Performs automatic feature selection.

- Disadvantages

- Prone to overfitting (especially deep trees).
- Unstable to small changes (can cause large changes in tree)
- Greedy algorithm $\rightarrow$ locally optimal splits, not always globally optimal.

- Root Nodes : It is the node present at the beginning of a decision tree. From this node, the population starts dividing according to various features.

- Decision Nodes : The nodes we get after splitting the root nodes are called Decision Nodes.

- Leaf Nodes : The nodes where further splitting is not possible are called leaf nodes or terminal nodes.

- Branch / Sub-tree : Sub-section of a decision tree is called sub-tree.

- Pruning : Cutting down some nodes to stop overfitting.

- Hyperparameters of a Decision Tree:

- Hyperparameters control how the tree is built, they're set before training and affect the model's performance, complexity and overfitting.

Criterion: decides how to measure the quality of a split. i.e. how it measures the best way to divide the data at each step.

- In sklearn, you can choose from:

- "gini" $\rightarrow$ Gini Impurity (default).

- "entropy" $\rightarrow$ Information gain (based on entropy)

- Gini Impurity:

- it measures the probability of misclassifying a randomly chosen sample if it were labeled according to the class distribution at a node.

- it measures how impure a node is, i.e., how many mixed categories are present at that node.

- If there's only one class at a node, it is pure node, gini impurity = 0.

Gini Impurity = $1 - \sum p_i^2$ and that the split
where p_i is the proportion of samples in class i .

- Decision will try a split for each feature and threshold. For each split, gini will be calculated. The split that has the least amount of Gini Impurity, that split will be chosen as the final.

Entropy: Entropy is the measure of uncertainty or randomness in the data.

- High Entropy means the data is mixed and confusing.
- Low Entropy means data is clear or pure.

$$\text{Entropy} = - \sum_{i=1}^c p_i \log_2(p_i)$$

c = number of classes

p_i = proportion of class i in the node.

- For each split, tree will calculate entropy for parent and child nodes. Information gain is calculated

$$\text{Info gain} = \text{Parent Entropy} - \text{Weighted Avg of Children Entropy}$$

- The split that has least entropy, i.e., the split that reduces uncertainty the most is used.

- Information Gain

- Information Gain is a measure used in decision trees (especially with criterion = "entropy") to choose the best feature for splitting the data at each step.

- Think of entropy as uncertainty or disorder.

Info gain measures how much that uncertainty drops after a split.

- High Info gain = better split.

$$\text{Info Gain} = H(\text{parent}) - \sum_{k=1}^K \frac{n_k}{n} \cdot H(\text{child}_k)$$

$H(\text{parent})$ = Entropy before the split.

$H(\text{child}_k)$ = Entropy of the k^{th} subset after the split.

n_k = number of samples in child k .

n = total samples in the parent.

max_depth :

- controls the maximum number of splits (levels) from root to leaf. In short, how deep your tree can grow.
- Too small depth (underfitting) : Tree is shallow, model too simple, can't capture patterns.
- Too large depth (overfitting) : Tree memorizes training data, performs poorly on test data.

min_sample_split :

- the minimum number of samples required to split an internal node.
- if a node has fewer than this number of samples, it won't be split.
- Helps prevent the tree from growing too deep (overfitting).
- Default = 2.

min_sample_leaf :

- the minimum number of samples that a leaf node must have.
- Even if a split is possible, it won't happen if it would leave a child with too few samples.
- Helps smooth out the model and reduce noise.
- Default = 1.

max-features :

- the number of features to consider when looking for the best split at each node.
- Randomly selecting a subset of features at each split can reduce overfitting, especially in ensemble models like Random Forests.

min-impurity-decrease :

- A node will only be split if the impurity decrease (information gain) caused by the split is greater than or equal to this threshold.
- prevents overfitting by not allowing small, less meaningful splits.
- Similar to pruning - helps keep the tree simpler and generalizable.

Performance Metrics

- Classification Metrics

1. Accuracy

- how well the model is performing in terms of correct predictions.

$$\text{Accuracy} = \frac{\text{No. of correct predictions}}{\text{Total no. of input samples.}}$$

$$= \frac{TP + TN}{TP + TN + FP + FN}$$

2. Precision

- how many positive predictions made by the model are actually correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

3. Recall

- ratio of correctly predicted positive instances to the total actual positive instances. It measures how well the model captures all relevant positive cases.

$$\text{Recall} = \frac{TP}{TP + FN}$$

4. F1 - Score

- it is the harmonic mean between recall and precision. Its range is $[0, 1]$.
- this metric usually tells us how precise (correctly classifies how many instances) and robust (does not miss any significant no. of instances) our classifier is.
- lower recall and higher precision give you great accuracy but it misses a large number of instances.
- the more the F1 score, better will be performance.

$$F1 = 2 \times \frac{1}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}}$$

5. Confusion Matrix

- it is a table used to evaluate the performance of a classification model by comparing model's predicted labels with actual labels.

		Predicted	
		No	Yes
Actual	No	TN	FP
	Yes	FN	TP

TP = True Positive = Predicted Yes, Actual also Yes.

TN = True Negative = Predicted No, Actual also No.

FP = False Positive = Predicted Yes, Actual No.

FN = False Negative = Predicted No, Actual Yes.

6. True Positive Rate

- also called sensitivity.

- proportion of positive data points that are correctly considered as positive, with respect to all data points that are positive.

$$TPR = \frac{TP}{TP + FN}$$

7. True Negative Rate

- also called specificity.

- a proportion of negative data points that are correctly considered as negative, with respect to all data points that are negatives.

$$TNR = \frac{TN}{TN + FP}$$

8. False Positive Rate

- proportion of actual positives that are incorrectly identified as negatives.

$$FPR = \frac{FP}{FP + TN}$$

$$\boxed{TPR + FPR = 1}$$

- lower threshold value will increase the Recall / ~~TPR~~ TPR/Sensitivity.

- higher threshold value will decrease the sensitivity.

$$\frac{91}{100} = 0.91$$

$$\frac{91}{100} = 0.91$$

also called as precision - a portion of positive data points that are correct when compared to threshold

$$\text{TPR} + \text{FPR} = 1$$

- lower threshold value will increase the Recall / ~~TPR~~ TPR/Sensitivity.

- higher threshold value will decrease the sensitivity.

$$\frac{\text{TP}}{\text{TP} + \text{FP}} = \text{TPR}$$

also called specificity - a fraction of negative data points correctly classified as negative

- Regression Evaluation Metrics

1. Mean Absolute Error (MAE)

- is the average distance between predicted and actual values.

$$MAE = \frac{1}{N} \cdot \sum_{i=1}^N |y_i - \hat{y}_i|$$

N = sample size

y_i = actual value

$\hat{y}_i$ = predicted value

- there is one limitation that it does not give any idea about the direction of the error which is whether we are under-predicting or over-predicting our data.

2. Mean Square Error (MSE)

- is similar to MAE but the difference is it takes the square of the average of between predicted and original values.

$$MSE = \frac{1}{N} \cdot \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- By taking square of errors, it pronounces larger errors more than smaller errors, we can focus more on larger errors.

3. Root Mean Square Error (RMSE)

- square root of MSE.

$$RMSE = \sqrt{MSE}$$

4. R² - Score

- also called coefficient of determination.
- it is the amount of variation in the output-dependent attribute which is predictable from the input independent variables.

- It is used to check how well-observed results are reproduced by the model, depending on the ratio of total deviation of results described by the model.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Naive Bayes Theorem

- assumes all features are independent, hence called 'naive'.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

$P(A|B)$ = prob^b A happening given B happened (Posterior)

$P(B|A)$ = prob B happening given A happened (Likelihood)

$P(A)$ = prob of A (Prior).

$P(B)$ = prob of B (evidence).

KNN - K Nearest Neighbours

- supervised type of machine learning
- lazy learner $\rightarrow$ it doesn't learn during training, it waits till you ask it something.
- KNN finds the 'k' closest data points (neighbours) to the one you're trying to predict.
- Then it checks their labels and does:
 - Majority voting for classification
 - Average for regression.
- Steps
 - choose value of k (odd numbers usually like 3, 5, 7).
 - calculate distance between test point and all training points.
 - Pick k nearest data points.
 - check labels. Return most frequent label (for classification) or average of training points (for regression).
- Distance Formula:
 - Euclidean

$$D = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$